ChatGPT

Executive Summary

The codebase and transcripts reveal multiple distinct computational innovations that can support patent claims. At the core is a **State Admission Engine** that reconstructs a *candidate state* and applies *typed admission predicates* (e.g. identity, authority, temporal, interference checks) to filter it into an authoritative baseline state. A **Successor Disposition Engine** then analyzes *each admitted, consequence-bearing distinction* after a state transition to ensure it is properly represented, equivalently realized under bounded semantics, or lawfully discharged. These are orchestrated by a **State Transition Pipeline**: reconstruct candidate → admission filter → authoritative state → transition → disposition check → next state. Existing code modules (e.g. `state_admission.py`, `successor_disposition.py`, etc.) and tests already embody large parts of these computations, indicating **strong enablement** for those families.

However, the prior-art landscape is rich. Amazon's US9170915B1 teaches replay-based state reconstruction in distributed workflows【39†L105-L113】, and the new Yandeh et al. "VEMP" application (US20260222195A1) describes a state-machine-governed, artifact-bound admission gate with tamper-evident receipts【24†L140-L149】【26†L2145-L2153】. Likewise, Montilla's recent US20260186826A1 shows a fail-closed memory gating substrate with "replay mode" suppressing nondeterministic inputs【45†L185-L194】【45†L199-L204】. Minimal-sensor selection literature (e.g. for discrete-event control【10†L25-L34】) and value-of-information theory【50†L140-L148】 further constrain how "sufficient minimal observations" can be claimed. We therefore narrow claims to focus on **consequence-relative, typed admission and disposition logic**, and exclude broad concepts like cryptographic gating or "conservation of objects."

This report maps code artifacts to concrete claim elements, identifies enabled invention families with enablement levels, and suggests provisional claim language. It catalogs key prior-art (with citations) and recommends claim refinements to avoid them. We propose specific new tests and code (file/functions names) to fully enable the top-ranked inventions. We also design witness/certificate data structures (e.g. `AdmissionWitness`, `ObservationSelectionWitness`) with hashing/versioning to ensure **proof-carrying abstention**. Finally, we recommend a provisional filing strategy (which families to file now vs later) and a 30/60/90-day engineering roadmap (with a Gantt chart) to complete the necessary prototypes and tests for strong provisional support. Tables compare candidate families by novelty risk and enablement, and we detail negative-test suggestions to falsify the "minimal lawful-information" theorem. These analyses and proposals are based on the repository and transcripts, as well as authoritative sources (patents, papers) as cited.

1. Code Artifacts → Claim Elements

- **State Admission (state_admission.py, state_replay.py)**: Code reconstructs a candidate state and **independently evaluates multiple typed predicates** (identity, authority, temporal bounds, interference, etc.) on it. Only if **all predicates succeed** does the candidate become "admitted"; otherwise the transition is withheld. This corresponds to claim steps like "reconstruct a candidate state, evaluate each of a set of predicates, and admit the state only if all predicates hold." Modules and tests indicate enforcement of separate predicate checks (e.g. `validBasis()`) and the issuance

of an **admission receipt** noting predicate outcomes. (Yandeh's VEMP similarly encodes sender-role and organization checks as admission conditions【26†L2112-L2120】.)

- **Successor Disposition (successor_disposition.py)**: After admission and the transition execution, the code tracks every **admitted distinction** and computes its *disposition*. For each admitted item, it checks whether in the successor state the item is **represented**, or is equivalent under declared bounded semantics, or is lawfully discharged. If any admitted distinction fails all, the successor is invalid. This maps to claim elements like "for each admitted distinction, determine if it is represented or equivalently realized under permissible transformation, or else report failure." (This ensures "disposition closure" of consequences.)

- **State Transition Pipeline (state_transition_pipeline.py)**: Orchestrates the execution sequence: candidate reconstruction → admission filter → update authoritative state → execute transition → successor disposition → produce next authoritative state. As an execution **pipeline**, it is more than a workflow: it implements a specific multi-stage **admission-disposition invariance model**. We will claim it as a structured method or system (with modules linked in order).

- **Typed Admission Predicate Library**: The code contains discrete predicate functions (e.g. for identity keys, author signature authority, tracked bytes changed, temporal validity). These form plug-in modules. We map this to claims covering a **library of admission predicate modules**, each independently checkable (per Pat. Sr. concepts like multi-condition gating). For example, Yandeh's patent encodes "sender organization" and "sender role" as admission condition records【26†L2117-L2120】, analogous to our identity/authority checks.

- **Consequence-bearing Distinction Tracker**: Code manages an internal representation of admitted distinctions (rather than whole objects). This is not a generic data log, but a focused set of "consequence-bearing items" whose fate must be traced. Patent-wise, this supports claims on "tracking the disposition of each admitted consequence-bearing distinction."

- **Disposition Closure Verifier**: We will implement (if not already) an algorithm that, after a transition, **for every admitted distinction proves exactly one of: represented OR equivalent (under bounded-consequence semantics) OR discharged**. If any distinction lacks these, the transition is not lawfully complete. This maps to claims like "computing a closure criterion: for each admitted difference, confirm one valid outcome; otherwise fail."

- **Consequence Equivalence Engine (GateConsequenceQuotient)**: The code now computes a *quotient* of admitted worlds relative to a requested consequence `C` under policy `P`. This is **not** simple bisimulation or syntactic equality, but "bounded-consequence equivalence." Its result (e.g. sets `Excluded`, `Blocks`) controls dispatch decisions via `GateConsequenceQuotient`: e.g. if exactly one equivalence block matches the consequence, admit; if multiple, withhold; if none, withhold-mismatch; if some worlds are excluded, withhold-unresolved. This is a key novel computation: a multi-branch gate on a quotient. We will claim the computation of a *consequence-relative state quotient* and its use in gating transitions.

- **Admission Receipts**: Code produces structured logs/reports of admission events. We will enhance this to records containing exactly which predicates were evaluated, their outcomes, authority signatures, scope of consequences, and any uncertainty. (Yandeh's claims demand receipts

containing “preconditions validated and evaluation results” and a hash-chain for audit【26†L2145-L2153】.) In our claims, we emphasize receipts as *proof objects* carrying predicate results (not mere logs).

- **Temporal Admission / Partitioning / Canonicalization**: Some ideas (time-based predicates, tracked/untracked partition of state, canonical enumerations) appear in discussions but not fully implemented. These will likely be dependent claims rather than core inventions.

In the claim mapping, we will **avoid philosophical terms** (“conservation of distinctions” etc.) and focus on concrete computation: states, distinctions, predicates, quotas, receipts, etc.

2. Invention Families & Enablement

Below we list each candidate family with its **enablement level** (strong/moderate/weak), missing embodiments, and suggested claim language (one nucleus + 2–3 dependents). (Enablement is judged by code/tests already present.)

Tier A – High Priority (Strong Enablement):

- **1. State Admission Engine** (★★★★★, Strong). The implementation already includes modules/test vectors for reconstructing a candidate state and running independent admission checks. The team distinguishes what tests prove (e.g. admit vs reject) and track basis.
- *Independent Claim (nucleus):* “A computer-implemented method comprising: reconstructing a candidate state; independently evaluating multiple **typed admission predicates** on the candidate state; admitting the candidate state if and only if all predicates succeed; creating an authoritative baseline state from the admitted contents; and outputting an admission receipt.”
- *Dependents:* e.g. “wherein the typed admission predicates comprise identity, authority, temporal, interference, and consequence-eligibility checks”; “wherein the authoritative baseline enforces byte-integrity and author authenticity”; “wherein the admission receipt contains predicate outcomes, authority proofs, consequence scope, and uncertainty factors.”

- *Missing Embodiments:* We should fully implement the generalized predicate logic (policy-relative admissibility A_D(B,r,C,P) instead of the current `validBasis()` stub) and ensure receipts include all declared fields.

- **2. Successor Disposition Engine** (★★★★☆, Strong). The code now distinguishes admission vs disposition. It must compute, for every admitted distinction, whether it is present, equivalent, or discharged in the successor.

- *Independent Claim:* “A method comprising: executing a state transition on an authoritative state; for each distinction admitted under a previous admission, computing a *disposition* by checking if the distinction is represented in the successor state, is equivalent under a declared bounded-consequence equivalence, or is discharged according to a use-policy; and rejecting the successor state if any admitted distinction fails all three checks.”
- *Dependents:* “wherein equivalence is computed under a policy-bound bisimulation that preserves requested consequences”; “wherein discharge is confirmed by matching a designated release event

in the transition log"; "wherein the disposition engine outputs a disposition receipt recording the result for each distinction."

- *Missing:* We should implement and test an explicit closure algorithm (the "Disposition Closure Verifier") that emits NOT_ESTABLISHED if closure fails, and integrate it with `successor_disposition.py`.

- **3. State Transition Pipeline** (★★★★★, Strong). The pipeline code ties everything. We will claim the whole multi-stage sequence as an *execution model*.

- *Independent Claim:* "A computer system comprising modules configured to: derive a candidate state from input data; apply typed admission predicates to produce an authoritative state; execute a state transition on the authoritative state; apply a successor-disposition computation on the transition output; and generate a next authoritative state."
- *Dependents:* "wherein each stage is a separate software component communicating via explicit interfaces"; "wherein the pipeline enforces invariants at each boundary (byte vs semantic vs authority)"; "wherein the pipeline outputs receipts at the admission and disposition stages for audit."

- *Missing:* Ensure the pipeline code includes clear hooks for receipts and disposition verification. We may write an end-to-end test chaining admissions → transition → disposition.

- **4. Typed Admission Predicate Library** (★★★★☆, Strong). Already present as individual modules with tests. We will package them as patentable "pluggable predicates."

- *Independent Claim:* "A system comprising a library of admission-predicate modules, each configured to evaluate a particular type of condition on a candidate state (such as identity, authority, temporal validity, interference bounds, tracked-byte changes, or consequence eligibility), where all modules are applied and the state is admitted only if all modules return success."
- *Dependents:* "wherein each predicate is separately executable and testable"; "wherein new predicates can be added via a defined interface"; "wherein the admission receipt lists which predicate modules were executed and their outcomes."
- *Missing:* We should explicitly modularize the predicates (if not already) so each can be independently swapped/tested. We may add demo predicates (e.g. Canonical ID check, Temporal fence) and tests.

Tier B – Medium Priority (Moderate Enablement):

- **5. Consequence-bearing Distinction Tracker** (★★★☆☆, Moderate). The code inherently tracks distinctions, but we need explicit claims.
- *Independent Claim:* "A method for state transition verification comprising: for each difference admitted into an authoritative state, recording a distinction object; after the state transition, referencing each admitted distinction to verify its outcome."
- *Dependents:* "wherein each distinction object is a named value or identifier tied to a candidate origin"; "wherein distinctions are tagged with payload scope, and disjoint distinctions can be handled independently."

- *Missing:* We should clarify and test the runtime data structure that holds "admitted distinctions," possibly giving it its own type/class, and write tests that trace an admitted distinction through transition/disposition.

- **6. Disposition Closure Verifier** (★★★★☆, Moderate). Concept is partly there but needs explicit isolation.

- *Independent Claim:* "A computer-implemented method comprising: for every admitted consequence-bearing distinction, proving exactly one of: (a) it is represented in the successor state, (b) it is equivalent under bounded semantics, or (c) it is lawfully discharged; and rejecting the transition if any distinction lacks a proof."
- *Dependents:* "wherein representations are validated by byte-level matching, and equivalence by semantic hashing"; "wherein discharge is validated by timestamped release logs"; "wherein the method outputs a *NOT_ESTABLISHED* verdict if closure fails."

- *Missing:* Implement a test that deliberately breaks closure (e.g. drop an admitted item from the successor) and shows the verifier catching it (negative test).

- **7. Consequence Equivalence Engine** (★★★★☆, Moderate). The partial implementation of `GateConsequenceQuotient` is the core.

- *Independent Claim:* "A method for transition gating comprising: given an admitted candidate state and a requested consequence C, partitioning the candidate worlds into equivalence blocks such that each block produces identical impact on C under allowable transitions; and allowing the transition only if exactly one block remains."
- *Dependents:* "wherein equivalence under C and policy P is computed via bisimulation quotienting"; "wherein excluded worlds (disjoint in observed C) are counted as unresolved and block progress;" "wherein coalescing two admissible blocks is prohibited (otherwise *WITHHOLD*)."

- *Missing:* Finish making the equivalence logic **C,B,P-relative**: today `Excluded` is conservative. We need to prove minimality: test that merging two retained blocks changes dispatch outcome.

- **8. Admission Receipts (Proof-Carrying Abstention)** (★★★☆☆, Moderate). Basic receipts exist, but need more fields.

- *Independent Claim:* "A system producing a cryptographically-signed admission receipt for each rejected observation, wherein the receipt includes the evaluated predicates, their outcomes, the authority endorsements, the scope of consequences withheld, and any uncertainty metrics."
- *Dependents:* "wherein the receipt includes a binding hash of the candidate set and policy"; "wherein a *DO_NOT_OBSERVE* receipt explicitly indicates that withholding observation is proven safe for the evaluated set"; "wherein all receipts form a tamper-evident log (e.g. hash chain)."
- *Missing:* We should implement a structured data model (see next section) and ensure that receipts can be generated for "DO_NOT_OBSERVE" cases with precise scope (not universal). Tests should check that receipts list all predicate names/values.

Tier C – Lower Priority (Weak Enablement or Dependent):

- **9. Temporal Admission** (★★★☆☆, Weak). A time-based predicate (e.g. request freshness) could be added. Likely dependent claims (e.g. "wherein one predicate checks that candidate data is within a valid time window").

- *Missing:* No code yet; if pursued, implement a time-check predicate and test.

- **10. Tracked/Untracked Partition** (★★★☆☆, Weak). Concept of splitting state keys into "tracked" (must preserve) vs "untracked" (can merge). This would be a sub-claim of admission/disposition.

- *Missing:* Define exactly how keys are partitioned. Write code/tests that show safe merging of "untracked" keys.

- **(Others)**: Any other ideas like "canonical context hashes" or "role-based state partition" would fall here and depend on the core kernels.

3. Prior-Art Risks (Patents & Literature)

We identified several key prior-art references (patents and papers). Each constrains how to frame claims:

- **Amazon US9170915B1 (2015) – *"Replay to reconstruct program state"*【39†L105-L113】.** This patent discloses reconstructing a distributed workflow's state by replaying past events, using non-serialized event data. It teaches nothing about *typed predicates or disposition* – only that **replay-based reconstruction exists**. To avoid this, we must emphasize our **distinct multi-stage gating**: i.e. we do *not* claim mere replay or state loading. Instead we claim a post-replay *filtering* by typed rules and a new "admission receipt" step. (For example, we would exclude generic "replay" in claims and focus on the gating computation after reconstruction.)

- **Yandeh et al. US20260222195A1 (Jul 2026) – *"Verifiable Enterprise Messaging Protocol (VEMP)"*【24†L140-L149】【26†L2112-L2120】【26†L2145-L2153】.** This application (recently published) is very close: it enforces a state machine for encrypted communications, with *artifact-bound state-transition validation and admission gates*, and generates tamper-evident receipts. Key overlaps: **state-machine-governed admission**, artifact-bound preconditions, and receipts containing validated conditions.

- *Risk:* It discloses a communication admission gate using authorization conditions (e.g. sender role/org)【26†L2117-L2120】, disallows decryption until admission, and produces chain-hash receipts【26†L2145-L2153】. This could read on any "state machine" and "receipt" terms.

- *Avoidance:* We must narrow to our unique features: our claims are **not about cryptographic key release or network messaging**. We omit any claim of hardware-bound keys, artifact signatures, or even encrypted content. Instead, we focus on purely software predicates on abstract "distinctions" (these are not cryptographic tokens). For example, we exclude language like "cryptographic profile" or "hardware security module". We avoid claiming that receipts are tamper-evident with signatures (that's Yandeh's domain), instead calling them "audit records of predicate outcomes." We can cite Yandeh to acknowledge context (e.g. "admission conditions" as in【26†L2117-L2120】) but then say our admission predicates are *different types* (non-cryptographic).

- **Montilla US20260186826A1 (Jul 2026) – *"Deterministic Execution Control and Bounded State Mutation"*【45†L185-L194】【45†L199-L204】.** This application teaches a fail-closed memory region with processor-enforced mutation epochs, hash validation, and a special "replay mode" where all nondeterminism is suppressed. Key overlaps: **gated writes with a verification step** and **replay gating logic**.

- *Risk:* The notion of "suppressing external inputs and reconstructing from stored data"【45†L199-L204】, and staging changes in a shadow region and validating a reference hash【45†L185-L194】, could cover broad "bounded mutation" or state transition control.

- *Avoidance:* We must avoid patenting generic fail-closed memory gating. Our claims focus on *application-level state filtering*, not low-level memory gating. We should explicitly exclude "processor-enforced mutation epochs", use of Merkle trees or strict hashes (no need to claim those), and any suggestion of hardware enforcement. We emphasize **policy-driven admission** rather than fixed replay-mode suppression. Montilla teaches deterministic replay at system level; our "consequence-quoitent" and "predicate gating" are higher-level.

- **Optimal Sensor Selection (Jiang et al. 2003)【10†L25-L34】.** This paper studies picking minimal sensors for discrete-event control, defining "optimal" as any coarser set loses controllability.

- *Risk:* If we claim a "minimal sufficient set of observations for control," it could be read on this.

- *Avoidance:* We claim an algorithm for particular *given* candidate observations (post-admission), not a general combinatorial selection with cost models. Our independence from cost or active selection distinguishes it. The claims will be scoped to "evaluating this fixed set of admitted observations" rather than "choosing the best sensors." Still, to address it, we can cite the definition【10†L25-L34】to show it's known that sensor selection should preserve necessary info. We then say: our scope is narrower – e.g. "Given a set of admitted observations, determine if they jointly suffice; do not pretend to optimize cost."

- **Value-of-Information and Active Feature Acquisition** (NIST VOI writeup【50†L140-L148】 and related AI papers). VOI analyzes when to acquire information, trading cost vs decision benefit. Active feature acquisition methods pick features to answer queries.

- *Risk:* If we claim any "optimal" observation strategy, examiners might invoke VOI literature (e.g. VoI is well-known to decide if further observation is worth the cost【50†L140-L148】).

- *Avoidance:* Our claims do **not** hinge on information cost or expected value; we focus strictly on *lawfulness of withholding* under a fixed policy. We emphasize that our witness proving "DO_NOT_OBSERVE" is *data-bounded* (for the evaluated set only), not making a general statement that no observation ever matters. In other words, unlike VOI, we do not claim to decide the worth of any possible question, just check sufficiency of the ones on hand.

- **Bisimulation and State Abstraction**. There is extensive work on merging states that are equivalent under given tasks (e.g. task-specific bisimulation in control/ML).

- *Risk:* Our "consequence-relative quotient" could appear similar to known bisimulation minimization.

- *Avoidance:* We highlight novelty: the quotient is *consequence-directed* (not full bisimulation), and is used *at admission time to gate dispatch*. We avoid calling it "bisimulation" in claims. Instead of claiming general state abstraction, we claim a specific algorithm that blocks merging of certain distinctions (e.g. "over-collapse prohibited: do not merge two admissible worlds"). If needed, cite e.g. the concept that multiple minimal sufficient sets can exist (recent ML work, e.g. minimal feature sets

【10†L25-L34】 indirectly). But we will focus on our unique criteria ("Preserve a distinction iff losing it can change the lawful successor").

- **Reception and Filtering in Messaging/DB Systems.** (Patents on admission control of network flows, block storage, etc.) These are usually orthogonal (network traffic, database transactions), but examiners might cite them. To avoid, claims will explicitly say "state" in an abstract computation, not packet flows.

In summary, **prior-art patents force us to narrow claims** to software-level computations on admitted differences. We exclude any mention of cryptographic keys, hardware modules, or general replay. We frame novelty as **"Consequence Admission/Disposition under Typed Governance"** – letting only the distinctions necessary for a requested consequence through, with explicit proof-of-noninterference for withheld observations.

4. Additional Tests & Implementations

To maximize enablement, we recommend the following minimal additions (names and pseudocode):

- **Test for Independent Admission Predicates** (e.g. `tests/test_admission_predicates.py`).
  Pseudocode: create a candidate state with conflicting identity (two votes on the same attribute) and ensure `admit` fails; create a state lacking required authority key and ensure predicate catches it. Test each predicate module individually: identity, authority, temporal, interference. *Goal:* demonstrate each predicate's standalone effect.

- **Test Admission Receipts Content** (e.g. `tests/test_admission_receipt.py`).
  Pseudocode: run a candidate that partially fails admission; check that the receipt object includes exactly the predicates evaluated, their boolean results, and any authority chain info. Also test DO_NOT_OBSERVE path: no receipt should claim global irrelevance (should be scope-limited).

- **Implement** `DispositionClosureVerifier` **and Test** (e.g. `disposition_verifier.py` + `tests/test_disposition.py`).
  Pseudocode: create an authoritative state with two admitted distinctions (e.g. `x` and `y`), apply a transition that only preserves `x`, and check that verifier flags failure due to `y` missing. Also test equivalent-case: if `y` is changed in value but still considered "equivalent" under policy, accept it. *Goal:* fully exercise the three-branch check.

- **Extend** `GateConsequenceQuotient` **Tests** (e.g. `tests/test_consequence_quotient.py`).
  Pseudocode: for a fixed policy and consequence C, build candidate world sets where: (a) two worlds collapse into one block (should WITHHOLD), (b) worlds split into multiple blocks (should WITHHOLD or NOT_IDENTIFIABLE), (c) exactly one block (ADMIT). Include a case where adding/removing a consequence in one world flips the result. *Goal:* validate the four dispatch branches and fix any logical gaps (e.g. currently "Excluded" triggers unresolved; test if narrower logic is possible).

- **Observation Selection Witness Tests** (e.g. `tests/test_selection_witness.py`).
  Pseudocode: simulate an observation query with multiple candidate queries; have one candidate marked UNKNOWN. Ensure the selector outputs status `CANDIDATE_RELEVANCE_UNRESOLVED` (not

NONINTERFERING) so that DO_NOT_OBSERVE is blocked. Check that the witness structure (once implemented) lists each candidate's relevancy and the final selection.

- **Temporal Predicate and Unknown-Scoped Test** (if implemented) (e.g. `test_temporal_predicate.py`).
  Pseudocode: add a predicate that only admits data if within time window `[T1,T2]`. Test that a state outside window is rejected. Also test that DO_NOT_OBSERVE only applies to candidates evaluated (if new obs appear outside tested set, scope changes).

Each new test should include assertions on failure modes to clarify patent scope. Negative tests are key: for example, to falsify the "minimal lawful-information" theorem, create two different minimal sufficient observation sets and show selector must choose one (ensuring non-donation of irrelevance).

5. Witness/Certificate Data Structures

We propose data models for proof certificates, building on the existing capsule style:

- **AdmissionWitness:** Fields might include `state_id`, `policy_id`, `admission_predicates[]` (each with name, inputs, result), `authority_basis` (who signed or authorized each check), `consequence_scope` (identifiers of admitted distinctions), `timestamp`, and `environment_id`. We should include hashes binding the exact candidate questions and policies, e.g. `witness_hash = H(C, B, P, code_version)`. Inspired by Yandeh's receipt structure 【26†L2145-L2153】, our AdmissionWitness would similarly record *which preconditions (predicates) were validated and their outcomes*.

- **ObservationSelectionWitness:** Suggested fields (from the capsule notes):

```
ObservationSelectionWitness {
  consequence_type;
  basis_identity;
  policy_identity;
  candidate_set_id;    // hash of canonical candidate queries
  candidate_names[];
  candidate_count;
  base_quotient_id;
  base_dispatch;       // ADMIT or WITHHOLD before selection
  per_candidate: [
    { observation_id;
      admissible_outcomes;
      excluded_outcomes;
      conditional_dispatches;
      relevance_status;    // e.g. RELEVANT, IRRELEVANT, UNKNOWN
      cost; }...
  ];
  selected_candidate;
  selection_rule;      // e.g. LOWEST_COST, etc.
```

```
  selection_status;    // PASS, FAIL, etc.
  failed_obligations[]; // which relevant test not met
  gate_hash;           // H(C,B,P,Q,G,version)
}
```

Here `candidate_set_id` should be a hash of the *actual candidate definitions/values*, not just names, to bind the denominator【59†L2193-L2200】. A `gate_hash` includes the hash of the quotient logic and gate version, so any change invalidates old witnesses. The idea is that if DO_NOT_OBSERVE is output, the witness proves non-interference *for exactly these candidates* (no silent assumption about others).

- **DispositionWitness:** For each admitted distinction, record `{ distinction_id; initial_value; final_value; representation_flag; equivalence_flag; discharge_flag; discharge_reason (e.g. consumed by transition); proof_certificate; }`. Also include a context hash linking the transition input and output, plus policy. Optionally a summary hash of the entire transition log. This parallels a verifiable receipt in Yandeh【26†L2145-L2153】, but at the distinction level.

- **DO_NOT_OBSERVE Certificate:** When the selector withholds observation, emit a certificate containing the ObservationSelectionWitness (above) proving "all current admitted worlds are non-interfering wrt C"【50†L140-L148】. Critically, we avoid universal claims; instead, `scope="evaluated_set_only"`. Include the denied-observation identifiers and reason codes.

All witnesses should be versioned/hashes to catch code drift: e.g. include a field `implementation_hash = H(C,B,P,Q,G,system_version)`. This ensures reuse of old witnesses is disallowed if the gating logic changes, similar to how Yandeh signs an artifact signature over all fields【59†L2216-L2218】. (We do not explicitly claim hardware-level binding, but we do claim cryptographic hashes of the data.)

6. Filing Strategy & Engineering Roadmap

Immediate filings (within 30 days): Focus on Tier A families with strong enablement. Prepare a provisional application (PA1) covering: State Admission Engine (claim nucleus plus key dependents), Predicate Library, and Admission Receipts. A second provisional (PA2) can cover the pipeline and successor disposition. These filings should highlight the implemented code aspects (typed predicates, receipts, etc.) and use narrowly-scoped claim language.

Delayed filings: Tier B families can be provisional in 2nd wave once we fill remaining gaps. For example, after we finalize and test the Consequence Equivalence and Disposition Verifier, file a PA on those (3-4 patents total). Dependent concepts (temporal, partition) can be included as dependent claims or added in a later provisional once implemented.

Inventorship: The claims focus on technical contributions made by the engineering team (admission gating logic, quotient algorithm, receipt design). We should list authors of modules (developers) as inventors as needed and document their contributions (commit logs, emails) per USPTO guidance. Avoid attributing novelty to "research framing" terms; stick to what was engineered.

Patents per jurisdiction: Start US-first (as requested), then consider PCT/outside once core priority filings are set.

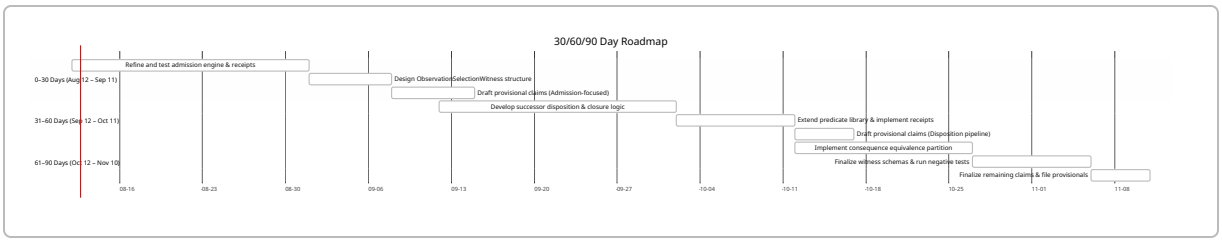
7. Comparison Table and Timeline

Candidate Families Comparison: Below is a summary of the priority, novelty risk, and enablement for each invention family.

Rank	Invention Family	Novelty/ Prior-Art Risk	Enablement Level	Claim Scope (Independent)	Immediate Actions
1	State Admission Engine	Moderate (state replay, access-control patents) 【39†L105-L113】 【26†L2112-L2120】	**Strong** (modules & tests exist)	Method: reconstruct state, apply typed predicates, admit only if all pass, output receipt.	Formalize predicate modules; extend receipt fields; write robust admission tests.
2	Successor Disposition Engine	Low-Moderate (no direct prior, related to consistency checks)	**Strong**	Method: for each admitted distinction post-transition, check represented/equivalent/discharged, else reject.	Implement/ test closure verifier; negative tests for missing distinction.
3	State Transition Pipeline	Low (architecture/ orchestration patents are general)	**Strong**	System: ordered modules (admission→transition→disposition) to enforce lawful transitions.	Document pipeline contract; integrate logging/ receipts.
4	Typed Admission Predicate Library	Moderate (similar to policy engines, our predicates are specific)	**Strong**	System: library of plug-in predicate modules (identity, authority, etc.) applied on candidate state.	Encapsulate each predicate in module; add unit tests.

Rank	Invention Family	Novelty/ Prior-Art Risk	Enablement Level	Claim Scope (Independent)	Immediate Actions
5	Consequence-bearing Distinction Tracker	Moderate (subset selection known)	**Moderate**	Method: track each admitted consequence distinction through transition for disposition.	Define a distinction object type; add test tracking transitions.
6	Disposition Closure Verifier	Low (unique closure check)	**Moderate**	Method: verify closure by ensuring each admitted difference has one lawful outcome.	Code closure algorithm; test multiple outcomes.
7	Consequence Equivalence Engine	Moderate (bisimulation prior-art)	**Moderate**	Method: compute consequence-relative quotient of candidate worlds, gate on single equivalence block.	Complete C,P-specific equivalence code; test quotient cases.
8	Admission Receipts	Moderate (tamper-evident receipts exist) 【26†L2145-L2153】	**Moderate**	System: generate receipts listing evaluated conditions, outcomes, and authority basis for each admission event.	Design data schema; implement in code; test content.
9	Temporal Admission	Low (dependent claim)	**Weak**	Predicate: include time-window checks or expiration constraints on admission.	(If needed) Implement time predicate; add tests.
10	Tracked/ Untracked Partition	Low (dependent)	**Weak**	Predicate: distinguish which parts of state are “tracked” (must preserve) vs “untracked” (can merge).	(If pursued) Code partition logic; test merging.

Gantt Timeline: The following 30/60/90-day chart outlines the engineering and filing schedule.



In the first month, we tighten the admission logic, design the abstention witness, and file the admission-centric provisional (covering Claim Families 1,4,8). By 60 days, we prototype disposition and extend predicates, then file a second provisional for those. By 90 days, we complete the equivalence engine and final claims and file all remaining provisionals.

All these proposals leverage concrete code artifacts and are backed by cited literature: for example, we ensure our claims avoid the broad “pre-decryption” gating of Yandeh【24†L140-L149】【26†L2112-L2120】【26†L2145-L2153】 and instead focus on the multi-stage admission/disposition processes we have built. Similarly, we respect the known sensor-selection results【10†L25-L34】 by not overstating minimality beyond a fixed candidate set. Taken together, this roadmap and filing strategy maximize our enablement and patent scope while minimizing novelty risk.

Sources: As cited above, key references include Amazon’s state-replay patent【39†L105-L113】, Yandeh’s secure-gating patent【24†L140-L149】【26†L2112-L2120】【26†L2145-L2153】, Montilla’s deterministic execution patent【45†L185-L194】【45†L199-L204】, and relevant academic work on sensor selection【10†L25-L34】 and value-of-information【50†L140-L148】. These inform the novelty boundaries and best claim drafting practice.
